# KloDock™

Desktop GUI for OpenClaw — zero terminal, zero complexity.

| | |
|---|---|
| **Version** | 1.1.0 |
| **Author** | Scott Converse |
| **License** | MIT |
| **Date** | March 2026 |
| **Repository** | github.com/scottconverse/klodock |

# Table of Contents


1 Overview

2 Key Features

3 Architecture

4 Security Model

5 Project Structure

6 Development Guide

7 Testing

8 Build Artifacts

9 CI/CD

10 Roadmap

11 Competitive Positioning

12 Known Limitations

13 License

Dashboard Screenshots

# 1 Overview

KloDock is a native desktop application that wraps the open-source OpenClaw AI agent framework in a visual, point-and-click interface. OpenClaw has amassed over 250,000 stars on GitHub and is one of the most capable open-source agent platforms available, but it demands terminal fluency, manual JSON and Markdown configuration, and comfortable API key management — skills that put it out of reach for most non-technical users. Meanwhile, hosted alternatives like Clawnify, OneClaw, and KiwiClaw charge between $10 and $40 per month for what is fundamentally free software, and they hold your API keys on their servers.

KloDock fills that gap. It packages the entire OpenClaw setup and management lifecycle into a guided wizard that takes minutes, not hours, from download to a running agent. There is no command line, no config file editing, and no Markdown to write. The user downloads a lightweight installer, runs the wizard, and talks to their agent. Everything stays local: keys are stored in the operating system's native credential store, the agent process runs on the user's machine, and no data leaves the device except what the user's chosen AI provider requires.

The application is built on Tauri v2, producing installers under 700 KB on Windows. It targets Windows, macOS, and Linux, with all three platforms passing CI builds and tests. KloDock is designed for accessibility from the ground up, meeting WCAG 2.1 AA requirements with full keyboard navigation, screen reader support, and high-contrast theming.

## 2 Key Features

**Quick setup wizard** — A 7-step guided flow that takes a new user from first launch to a running AI agent in minutes, not hours. No terminal, no manual steps.

**Silent Node.js installation** — Automatically detects existing Node.js installs, nvm, Volta, and Homebrew-managed versions. If Node.js is missing or below v22, KloDock silently downloads v24.14.0 LTS and extracts it to `~/.klodock/node/` with no elevation prompt.

**Visual personality builder** — Choose from role templates (assistant, creative writer, coder, etc.), adjust a tone slider from formal to casual, and watch a live SOUL.md preview update in real time.

**Secure API key management** — Keys are stored in the OS credential store: DPAPI on Windows, Keychain on macOS, libsecret on Linux. Keys never exist in plaintext at rest.

**Ollama integration** — Auto-detection of a local Ollama instance, model listing, model picker dropdown, no-models guard, and automatic configuration. **Ollama requires a model that supports tool calling.** Recommended models:

| MODEL | SIZE | NOTES |
|---|---|---|
| `qwen2.5:7b` | 4.7 GB | Good balance of speed and capability |
| `llama3.1:8b` | 4.7 GB | Meta's flagship small model with tool support |
| `mistral:7b` | 4.1 GB | Fast inference, solid tool calling |

**Channel setup** — Guided configuration flows for Telegram and Discord, with token entry, validation, and connection testing built into the wizard.

**Skill marketplace with safety ratings** — Browse and install skills from ClawHub with one click. Every skill carries a safety rating: Verified, Community, or Unreviewed.

**Agent lifecycle management** — The OpenClaw agent runs as a managed child process with

automatic restart on crash, periodic health monitoring, and graceful shutdown.

---

**Start on login** — Optional system tray launcher. Uses a registry key (Windows), launch agent (macOS), or XDG autostart entry (Linux).

---

**Resumable setup and uninstall** — Both flows persist progress to disk as JSON. If the app crashes mid-operation, it picks up where it left off.

---

**WCAG 2.1 AA accessible** — Full keyboard navigation, ARIA labels, screen reader testing, and high-contrast support throughout.

---

**Lightweight** — Windows MSI: 672 KB. NSIS setup: 415 KB. Leverages the system webview rather than bundling a browser engine.

---

**Full management dashboard** — Six dedicated pages (Overview, Skills, Personality, Channels, Settings, Updates) replace all wizard placeholders.

---

**52 categorized skills** — Skills grouped into 8 categories with search, filtering, and actionable setup buttons.

---

**Auto-updater** — Checks npm registry for the latest OpenClaw version and offers one-click update.

---

**Toast notification system** — Global success, error, warning, and info notifications. Auto-dismiss with manual close.

---

**Uninstall from UI** — Danger Zone section in Settings with full 7-step resumable uninstall and two confirmation dialogs.

# 3 Architecture

## System Architecture Diagram

```
+-------------------------------------------------------------+
|                       KloDock (Tauri v2)                    |
|                                                             |
|  +-------------------------+   +-------------------------+ |
|  |    React Frontend       |   |    Rust Backend         | |
|  |                         |   |                         | |
|  |  Wizard (7 steps)       |   |  installer/             | |
|  |  Dashboard (6 pages)    |<->|    node.rs              | |
|  |  Components             |IPC|    openclaw.rs           | |
|  |  Lib (types, state)     |   |    skills.rs            | |
|  |                         |   |    uninstall.rs          | |
|  +-------------------------+   |  config/                 | |
|                                |    soul.rs               | |
|                                |    openclaw_json.rs      | |
|                                |    env.rs                | |
|                                |  secrets/                | |
|                                |    keychain.rs           | |
|                                |  setup/                  | |
|                                |    setup_state.rs        | |
|                                |  process/                | |
|                                |    daemon.rs             | |
|                                |    health.rs             | |
|                                |    autostart/            | |
|                                |  clawhub/                | |
|                                |    registry.rs           | |
|                                |    safety.rs             | |
|                                |  update/                 | |
|                                |    openclaw_update.rs    | |
|                                |    skill_update.rs       | |
|                                +-------------------------+ |
+-------------------------------------------------------------+
         |                               |
         v                               v
  +-------------+                +------------------+
  | System      |                | ~/.klodock/      |
  | Webview     |                |   node/          |
  | (WRY/Edge/  |                |   secrets/       |
  |  WebKitGTK) |                |   setup-state.json|
  +-------------+                +------------------+
                                          |
                                          v
                                 +------------------+
                                 | OpenClaw Agent   |
                                 | (child process)  |
                                 |   ~/.openclaw/   |
                                 |     .env         |
                                 |     SOUL.md      |
```

```
|     openclaw.json|
+------------------+
```

## Why Tauri v2

1. **Bundle size.** Tauri leverages the OS native webview instead of shipping Chromium. Installer under 700 KB vs. 80–150 MB for Electron.
2. **Memory footprint.** Baseline ~30–50 MB vs. 150–300 MB for Electron, important since KloDock runs alongside the agent process.
3. **Security.** Capability-based permission model restricts the frontend to explicitly registered IPC commands. No Node.js in the main process.

## Rust Backend Modules

| MODULE | FILES | PURPOSE |
|---|---|---|
| `installer/` | node.rs, openclaw.rs, skills.rs, uninstall.rs | Downloads, extracts, and verifies Node.js; installs OpenClaw from npm; installs skills from ClawHub; manages uninstall lifecycle |
| `config/` | soul.rs, openclaw_json.rs, env.rs | Reads and writes SOUL.md, openclaw.json, and the ephemeral .env file |
| `secrets/` | keychain.rs | OS credential store integration — DPAPI / Keychain / libsecret |
| `setup/` | setup_state.rs | Wizard state persistence, step completion tracking, crash-safe resume |
| `process/` | daemon.rs, health.rs, autostart/ | Child process lifecycle, periodic health checks, platform-specific login-item registration |
| `clawhub/` | registry.rs, safety.rs | Skill registry client, search, recommendations, and safety rating lookup |
| `update/` | openclaw_update.rs, skill_update.rs | Version checking and update logic for OpenClaw and installed skills |

## Frontend Stack

| TECHNOLOGY | VERSION | ROLE |
|---|---|---|
| React | 19 | UI framework |
| TypeScript | 5.9 | Type safety |
| Tailwind CSS | v4 | Utility-first styling |
| React Router | v7 | Client-side routing |
| Lucide React | 0.577 | Icon library |
| Vite | 8 | Build tooling and dev server |
| Vitest | 4.1 | Unit and component testing |

# 4 Security Model

KloDock treats API keys as the most sensitive data in the system. The security model is designed around the principle that keys should never exist in plaintext at rest.

> **Note:** Users who choose Ollama (local AI) skip the entire key management flow. No API key is stored, no `.env` is materialized, and no credentials exist anywhere. The agent talks directly to Ollama at `http://localhost:11434`.

## Secret Materialization Flow (Cloud Providers)

1. User enters API key in the wizard UI
2. Frontend sends key to Rust backend via Tauri IPC
3. Rust backend encrypts and stores in OS credential store (DPAPI on Windows, Keychain on macOS, libsecret on Linux)
4. Key stored at rest (encrypted by OS, never plaintext)
5. *When daemon starts:* Rust backend retrieves key from credential store
6. Writes ephemeral `.env` file to `~/.openclaw/.env` with permissions set to 600 (owner only)
7. OpenClaw agent process reads `.env` on startup
8. *When daemon stops:* `.env` file is scrubbed (deleted) immediately

## Additional Security Measures

- **Crash recovery scrub.** On every KloDock launch, the Rust backend checks for and removes any stale `.env` file left behind by a prior crash.
- **SHA256 checksum verification.** The Node.js download is verified against the official `SHASUMS256.txt` from nodejs.org before extraction.
- **Windows file permissions.** The `.env` file and secret storage directory are locked to the current user via `icacls`.
- **Tauri capability restrictions.** The frontend can only invoke IPC commands explicitly registered in the `invoke_handler`. No shell access, no arbitrary filesystem access, no

network access from the webview context.

- **Ollama zero-key path.** No API key stored, no `.env` file created, no credentials leave the machine. Requires a tool-calling capable model.
- **No telemetry.** KloDock collects and transmits no usage data by default.

## 5 Project Structure

```
klodock/
├── .github/workflows/
│   ├── build.yml           # Cross-platform build
│   ├── test.yml            # Cross-platform test suite
│   ├── release.yml         # Tag-triggered release
│   └── compat.yml          # Nightly compatibility check
├── e2e/
│   ├── wizard-walkthrough.e2e.ts   # 8 wizard E2E tests
│   └── accessibility.e2e.ts        # 7 accessibility E2E tests
├── src/                        # React frontend
│   ├── __tests__/          # Unit & component tests
│   ├── components/         # Shared UI components
│   ├── dashboard/          # 6 dashboard pages
│   ├── lib/                # Types, state, utilities
│   ├── wizard/             # 8 wizard screens
│   └── App.tsx             # Root component + router
├── src-tauri/
│   ├── src/
│   │   ├── installer/      # Node, OpenClaw, skills, uninstall
│   │   ├── config/         # soul.rs, openclaw_json.rs, env.rs
│   │   ├── secrets/        # keychain.rs (DPAPI/Keychain/libsecret)
│   │   ├── setup/          # Wizard state persistence
│   │   ├── process/        # Daemon, health, autostart
│   │   ├── clawhub/        # Skill registry + safety
│   │   └── update/         # Version check + update
│   └── tests/              # Rust integration tests
├── package.json
├── vite.config.ts
└── tsconfig.json
```

# 6 Development Guide

## Prerequisites

| REQUIREMENT | DETAILS |
|---|---|
| Node.js | >= 22 (v24.14.0 LTS pinned) |
| Rust | Stable toolchain (rustup.rs) |
| Windows | Visual Studio Build Tools with C++ workload |
| macOS | Xcode Command Line Tools |
| Linux (Ubuntu) | `libwebkit2gtk-4.1-dev`, `libappindicator3-dev`, `librsvg2-dev` |

## Initial Setup

```
git clone https://github.com/scottconverse/klodock.git
cd klodock
npm install
```

## Development Server

```
npx tauri dev
```

This compiles the Rust backend, starts the Vite dev server for the React frontend, and opens the application window. Changes to `.tsx` files trigger instant HMR; changes to `.rs` files trigger a Rust recompile.

## Code Structure Conventions

- **Frontend IPC calls** go through `src/lib/tauri.ts`, wrapping `@tauri-apps/api/core.invoke()` with typed signatures.
- **Rust commands** are `#[tauri::command]` async functions registered in `lib.rs`.

- **Shared types** are defined in `src/lib/types.ts` (frontend) and as `serde`-derived structs in Rust.

## Useful Commands

| COMMAND | PURPOSE |
|---|---|
| `npx tauri dev` | Dev server with hot-reload |
| `npx vitest run` | Run frontend tests once |
| `npx vitest` | Run frontend tests in watch mode |
| `cd src-tauri && cargo test` | Run Rust tests |
| `cd src-tauri && cargo test -- --ignored` | Run ignored Rust tests |
| `npm run test:e2e` | Run E2E tests |
| `npx tauri build` | Production build with installers |

# 7 Testing

## Test Summary

| LAYER | FRAMEWORK | COUNT | STATUS |
|---|---|---|---|
| Frontend unit/component | Vitest + Testing Library | 26 | Passing |
| Rust integration | Cargo test | 12 | Passing |
| Rust (ignored, require real state) | Cargo test --ignored | 7 | Passing |
| End-to-end | WebdriverIO v9 + tauri-driver | 13 | Passing |
| **Total** | | **58** | **0 failures** |

## Frontend Tests (Vitest)

Tests in `src/__tests__/` cover component tests (ToneSlider, SafetyBadge), wizard screen tests (ModelProvider, Personality), and library tests (wizard-state).

## Rust Integration Tests (Cargo)

Tests in `src-tauri/tests/` cover Node.js detection, version manager detection, setup state persistence, daemon lifecycle, uninstall state, and autostart.

## Ignored Tests (Require Real System State)

Marked `#[ignore]` because they interact with real OS resources: keychain store/retrieve/delete (3 tests), full Node.js download (2 tests), autostart enable/disable (1 test per platform), and verify_all_steps with real system checks (1 test).

## End-to-End Tests (WebdriverIO)

13 real E2E tests launch the compiled application using WebdriverIO v9 with tauri-driver and msedgedriver. Includes 8 wizard walkthrough tests and 7 WCAG 2.1 AA accessibility tests.

# 8 Build Artifacts

## What `npx tauri build` Produces

| PLATFORM | ARTIFACT | FORMAT | NOTES |
|---|---|---|---|
| Windows | MSI installer | `.msi` | Standard Windows Installer package |
| Windows | NSIS installer | `.exe` | Lighter-weight NSIS setup executable |
| macOS | App bundle | `.app` in `.dmg` | Universal binary (Intel + Apple Silicon) |
| Linux | AppImage | `.AppImage` | Portable, no install required |
| Linux | Debian package | `.deb` | For apt-based distributions |

## Verified Windows Build Sizes

| ARTIFACT | SIZE |
|---|---|
| `KloDock_1.1.0_x64_en-US.msi` | 672 KB |
| `KloDock_1.1.0_x64-setup.exe` | 415 KB |

Output directory: `src-tauri/target/release/bundle/`

# 9 CI/CD

Four GitHub Actions workflows automate building, testing, releasing, and compatibility monitoring.

| WORKFLOW | FILE | TRIGGER | PLATFORMS | PURPOSE |
|---|---|---|---|---|
| **Build** | build.yml | Push/PR to main | Ubuntu, macOS, Windows | Compiles full Tauri application on all platforms |
| **Test** | test.yml | Push/PR to main | Ubuntu, macOS, Windows | Runs Rust, frontend, and E2E tests |
| **Release** | release.yml | Tag push (v*) | Ubuntu, macOS, Windows | Builds installers and publishes GitHub Release assets |
| **Compatibility** | compat.yml | Nightly (06:00 UTC) | Ubuntu | Tests against latest OpenClaw; auto-creates issues on failure |

## Dependency Caching

All workflows cache Cargo registry and build artifacts, plus `node_modules`. Cache keys are based on lockfile hashes.

# 10 Roadmap

## Phase 1: Setup Wizard SHIPPED

- 8-screen setup wizard from download to running agent
- Silent Node.js detection and installation
- OpenClaw installation from npm
- API key management with OS credential store
- Personality builder with SOUL.md preview
- Channel configuration (Telegram, Discord)
- Skill browser with safety ratings
- Agent lifecycle management
- System tray autostart
- Resumable setup and uninstall
- Accessibility (WCAG 2.1 AA)
- Cross-platform CI/CD with nightly compatibility checks

## Phase 2: Dashboard and Ecosystem V1.1.0

- ~~Agent dashboard~~ — **SHIPPED**
- ~~Skill marketplace~~ — **SHIPPED**
- ~~Settings panel~~ — **SHIPPED**
- WhatsApp integration — Deferred, evaluating alternatives
- Multi-agent support — Run and manage multiple instances

## Phase 3: Future Considerations PLANNED

- ~~Auto-update~~ — **SHIPPED in v1.1.0**

- Plugin system — Community-developed UI extensions
- Team management — Shared configurations for organizations
- Analytics dashboard — Usage metrics and insights (local only)
- Mobile companion — Status monitoring from mobile devices

# 11 Competitive Positioning

```
                    More Control / Local

                          * KloDock
                    (free, local, GUI)

Easier ----                          ---- Harder

      Clawnify      OneClaw      KiwiClaw
     ($$$)         ($10/mo)     ($15-39/mo)

                     OpenClaw CLI
                (free but terminal)

                    Less Control / Cloud
```

| PRODUCT | PRICE | LOCAL/CLOUD | GUI | TERMINAL REQUIRED |
|---|---|---|---|---|
| **KloDock** | **Free** | **Local** | **Yes** | **No** |
| OpenClaw CLI | Free | Local | No | Yes |
| Clawnify | $20+/mo | Cloud | Yes | No |
| OneClaw | $10/mo | Cloud | Yes | No |
| KiwiClaw | $15–39/mo | Cloud | Yes | No |

KloDock occupies the unique position of being both free and easy to use while keeping all data local.

## 12 Known Limitations

- **OpenClaw is fictional.** This project is a proof-of-concept demonstrating the architecture, UI patterns, and security model for wrapping a complex CLI tool in a native desktop GUI.
- **WhatsApp integration deferred.** Deferred because the Baileys library is fragile and breaks frequently with WhatsApp protocol changes.
- **No code signing.** Windows and macOS builds are not code-signed, which will trigger OS security warnings on first launch.

# 13 License

**MIT License**

Copyright (c) 2026 Scott Converse

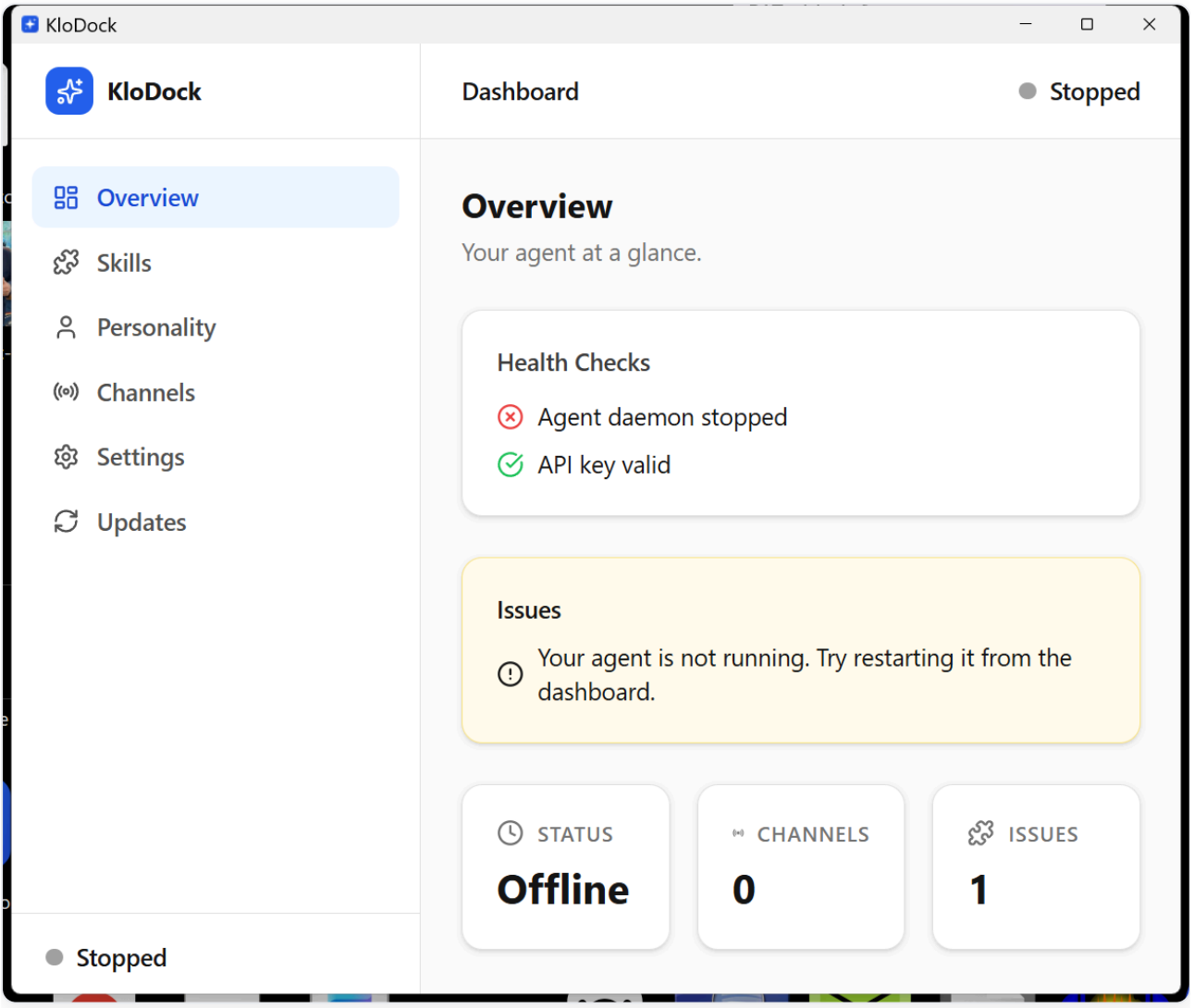
Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

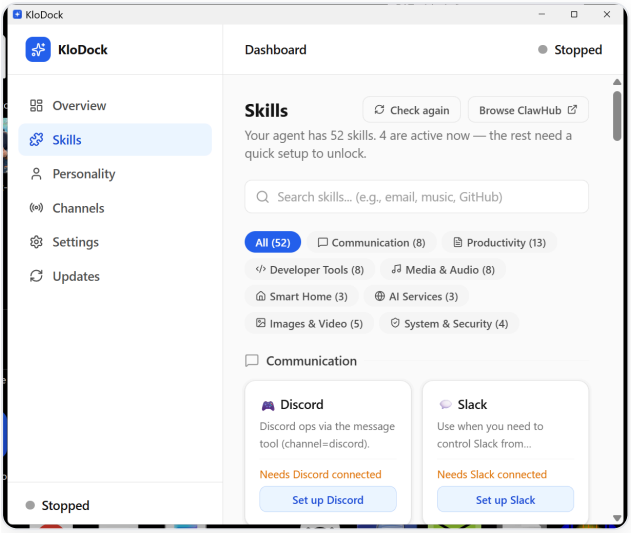
THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

# Dashboard Screenshots

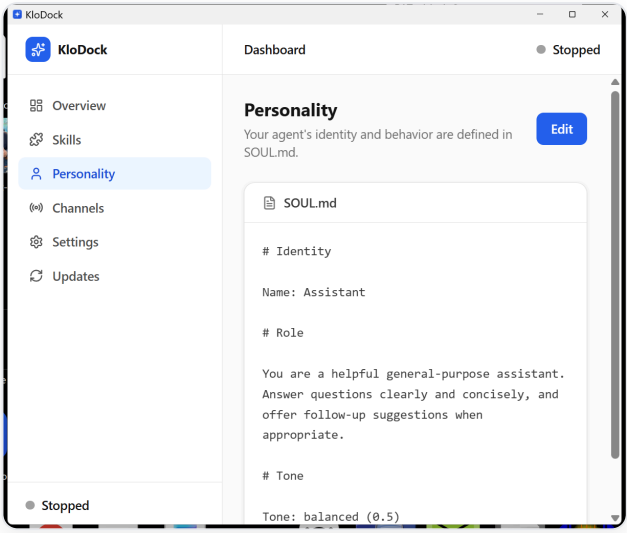
## Overview



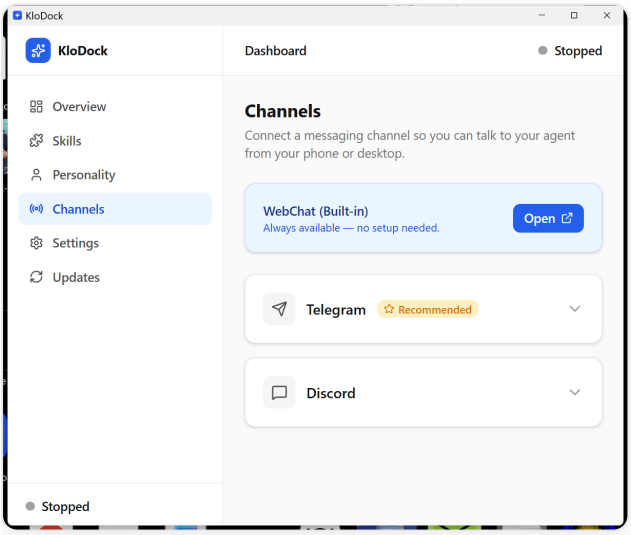
## Skills



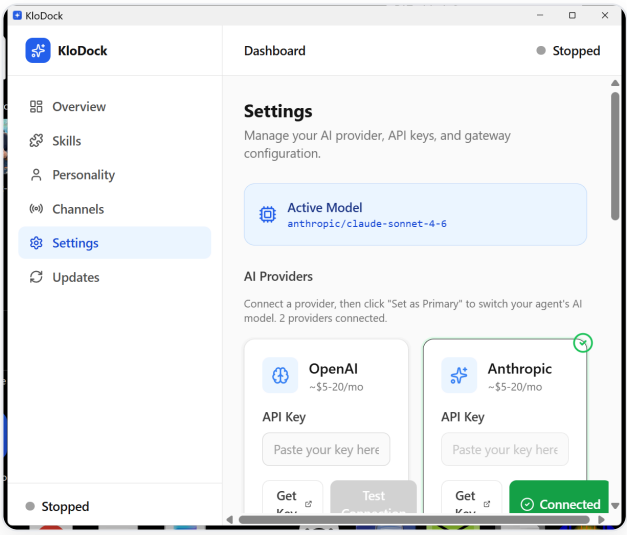

## Personality



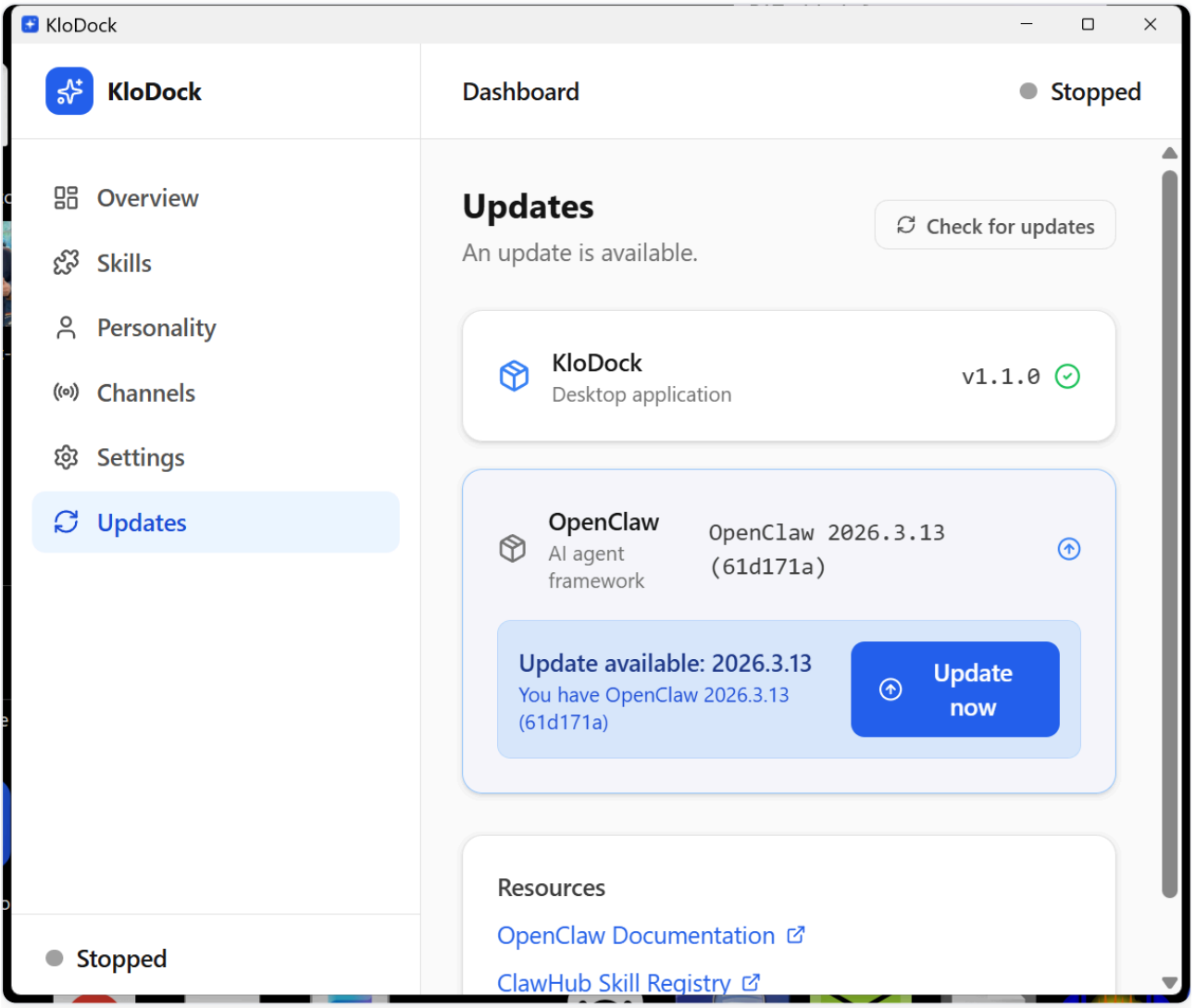

## Channels




## Settings



## Updates


KloDock
KloDock
Dashboard
Stopped
Overview
Skills
Personality
Channels
Settings
Updates
Stopped
Updates
An update is available.
Check for updates
KloDock
Desktop application
v1.1.0
OpenClaw
AI agent framework
OpenClaw 2026.3.13 (61d171a)
Update available: 2026.3.13
You have OpenClaw 2026.3.13 (61d171a)
Update now
Resources
OpenClaw Documentation
ClawHub Skill Registry